

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07
NAME: G. SINDHU
REG.NO: 192424197

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate

the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

ANSWERS

Question 1: Star Topology Generator

Aim:

Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. Display the topology in ASCII format, print the host label, switch name, and Cat6 UTP cable length for each connection. Verify all hosts are connected and explain the reliability of star topology.

Algorithm:

1. Start.
2. Read the number of hosts from the user.
3. Create the central switch named Switch1.
4. For each host:
 - Assign a host name (Host1, Host2, ...).
 - Read the Cat6 UTP cable length.
 - Display the host connected to the switch.
5. Verify that every host is connected to Switch1.
6. Print the explanation of Star Topology.
7. Stop.

Program:

```
n = int(input("Enter number of hosts: "))
switch = "Switch1"

for i in range(1, n + 1):
    length = int(input(f"Enter cable length for Host{i} (m): "))
    print(f"Host{i} ---- {length}m Cat6 UTP ----> {switch}")

print("\nVerification:")
print("All hosts are connected through", switch)

print("\nStar Topology:")
print("Each host is connected to the central switch.")
print("If one cable fails, other hosts continue to work.")
```

Sample Input:

Enter number of hosts: 3
Enter cable length for Host1 (m): 10
Enter cable length for Host2 (m): 15
Enter cable length for Host3 (m): 20

Output:

Host1 ---- 10m Cat6 UTP ----> Switch1
Host2 ---- 15m Cat6 UTP ----> Switch1
Host3 ---- 20m Cat6 UTP ----> Switch1

Verification:

All hosts are connected through Switch1

Star Topology:

Each host is connected to the central switch.

If one cable fails, other hosts continue to work.

Question 2: UTP Cable Validation

Aim:

To develop a Python function `validate_segment(length_m)` that checks whether a UTP cable segment satisfies the IEEE 802.3 Ethernet standard (maximum cable length of **100 meters**).

Algorithm:

1. Start.
2. Define a function `validate_segment(length_m)`.
3. If the cable length is less than or equal to 100 meters:
 - Return True.
4. Otherwise:
 - Return False and display an error message.
5. Test the function with three cable lengths.
6. Stop.

Program:

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True  
    else:  
        print("Error: Cable length exceeds 100 meters.")  
        return False  
  
# Test Cases  
print("Test Case 1:", validate_segment(50))  
print("Test Case 2:", validate_segment(100))  
print("Test Case 3:", validate_segment(120))
```

Output:

```
Test Case 1: True  
Test Case 2: True  
Error: Cable length exceeds 100 meters.  
Test Case 3: False
```

Interpretation:

- **50 m** → Valid cable length (**True**)
- **100 m** → Valid cable length (**True**)
- **120 m** → Invalid cable length (**False**) because it exceeds the IEEE 802.3 limit of **100 meters**.

Question 3: CSMA/CD Simulation

Aim:

To develop a Python program that simulates the **CSMA/CD (Carrier Sense Multiple Access with Collision Detection)** protocol for four hosts connected through a central switch, records transmission attempts, collisions, and back-off intervals in a log file (csma_log.txt)

Algorithm:

1. Start.

2. Create four hosts.
3. Randomly simulate whether a collision occurs.
4. If a collision occurs:
 - Generate a random back-off time.
 - Record the event in csma_log.txt.
5. Otherwise, record successful transmission.
6. Repeat for all hosts.
7. Display the simulation result.
8. Stop.

Program:

```
import random

hosts = ["Host1", "Host2", "Host3", "Host4"]

log = open("csma_log.txt", "w")

for host in hosts:
    print(host, "attempting transmission...")
    log.write(host + " attempting transmission\n")

    if random.choice([True, False]):
        backoff = random.randint(1, 5)
        print("Collision detected! Back-off:", backoff, "seconds")
        log.write(f"Collision detected. Back-off = {backoff} seconds\n")
    else:
        print("Transmission Successful")
        log.write("Transmission Successful\n")

log.close()

print("\nSimulation Completed.")
print("Log saved in csma_log.txt")
```

Sample Output:

```
Host1 attempting transmission...
Transmission Successful

Host2 attempting transmission...
Collision detected! Back-off: 3 seconds
```

Host3 attempting transmission...
Transmission Successful

Host4 attempting transmission...
Collision detected! Back-off: 2 seconds

Simulation Completed.
Log saved in csma_log.txt

Sample csma_log.txt

Host1 attempting transmission
Transmission Successful
Host2 attempting transmission
Collision detected. Back-off = 3 seconds
Host3 attempting transmission
Transmission Successful
Host4 attempting transmission
Collision detected. Back-off = 2 seconds

Explanation

- Four hosts attempt to transmit data.
- If two transmissions occur simultaneously, a **collision** is detected.
- A **random back-off interval** is generated before retrying.
- All transmission events are stored in **csma_log.txt**.
- CSMA/CD reduces data transmission conflicts by detecting collisions and delaying retransmission, improving Ethernet communication.

Question 4: Client-Server Communication using Socket

Aim:

To develop a Python client-server application using Python's **socket** library to simulate communication between two hosts connected through a central switch and display the updated **MAC Address Table** after successful frame transmission.

Algorithm:

Server

1. Start the server.

2. Create a socket and bind it to localhost.
3. Listen for a client connection.
4. Receive the frame from the client.
5. Store the client's MAC address in the MAC table.
6. Display the received frame and MAC table.
7. Close the connection.

Client

1. Create a socket.
2. Connect to the server.
3. Send the frame.
4. Close the connection.

Server Program (server.py)

```
import socket

server = socket.socket()
server.bind(("localhost", 5000))
server.listen(1)

print("Server waiting...")

conn, addr = server.accept()

data = conn.recv(1024).decode()

mac_table = {"AA:11:22:33:44:55": "Port1"}

print("Received Frame:", data)

print("\nMAC Address Table")
print("-----")
for mac, port in mac_table.items():
    print(mac, "->", port)

conn.close()
server.close()
```

Client Program (client.py)

```
import socket

client = socket.socket()
client.connect(("localhost", 5000))
```

```
frame = input("Enter Frame: ")
client.send(frame.encode())
```

```
client.close()
```

Sample Input:

Client

Enter Frame: Hello Switch

Server Output:

Server waiting...

Received Frame: Hello Switch

MAC Address Table

AA:11:22:33:44:55 -> Port1

Explanation:

- The **client** sends a frame to the **server**.
- The **server** receives the frame and updates the **MAC Address Table**.
- **MAC Learning:** The switch learns the sender's MAC address and associates it with the incoming port.
- **Frame Forwarding:** The switch forwards the frame only to the destination port if the destination MAC address is known; otherwise, it broadcasts the frame to all ports except the source.

Question 5: StarSwitch Class

Aim:

To design and implement a Python class **StarSwitch** that models the basic functionality of an Ethernet switch by adding ports, removing ports, and broadcasting frames to all connected hosts except the source host.

Algorithm:

1. Start.
2. Create a class StarSwitch.
3. Add hosts using add_port().
4. Remove hosts using remove_port().
5. Broadcast a frame using broadcast().

6. Display active ports and broadcast details.
7. Stop.

Program:

```
class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)
        print(host, "connected")

    def remove_port(self, host):
        self.ports.remove(host)
        print(host, "disconnected")

    def broadcast(self, source):
        print("\nBroadcast Message")
        for host in self.ports:
            if host != source:
                print(source, "->", host)

# Main Program
s = StarSwitch()

s.add_port("Host1")
s.add_port("Host2")
s.add_port("Host3")
s.add_port("Host4")

print("\nActive Ports:", s.ports)

s.broadcast("Host1")

s.remove_port("Host4")

print("\nActive Ports:", s.ports)
```

Sample Output:

```
Host1 connected
Host2 connected
```

Host3 connected

Host4 connected

Active Ports: ['Host1', 'Host2', 'Host3', 'Host4']

Broadcast Message

Host1 -> Host2

Host1 -> Host3

Host1 -> Host4

Host4 disconnected

Active Ports: ['Host1', 'Host2', 'Host3']

Explanation:

- `add_port()` connects a host to the switch.
- `remove_port()` disconnects a host from the switch.
- `broadcast()` sends a frame from the source host to **all other connected hosts**.
- Broadcasting is important in Ethernet for sending messages (such as ARP requests) to all devices on the network except the sender.